End-to-End DevOps CI/CD Pipeline Implementation

Table of Contents

1. **Project Overview and Architecture**
2. **Source Code Management (GitHub)**
3. **Jenkins Master Setup (Docker)**
4. **Jenkins Agent Setup (GCP VM)**
5. **SonarQube Setup and Jenkins Integration**
6. **Code Quality & Security Tool Setup**
7. **Docker Image Build & Tagging**
8. **Push Docker Image to Docker Hub**
9. **Push Docker Image to GCP Artifact Registry**
10. **GKE Cluster Setup & ArgoCD GitOps Deployment**
11. **Jenkins CI/CD Pipeline – DevSecOps Workflow**
12. **Monitoring Setup – Prometheus & Grafana (GKE)**
13. **Conclusion**

1. Project Overview and Architecture :-

Objective

To design and implement an end-to-end DevSecOps CI/CD pipeline that automates application build, testing, security scanning, containerization, deployment, and monitoring using modern cloud-native tools.

Project Description

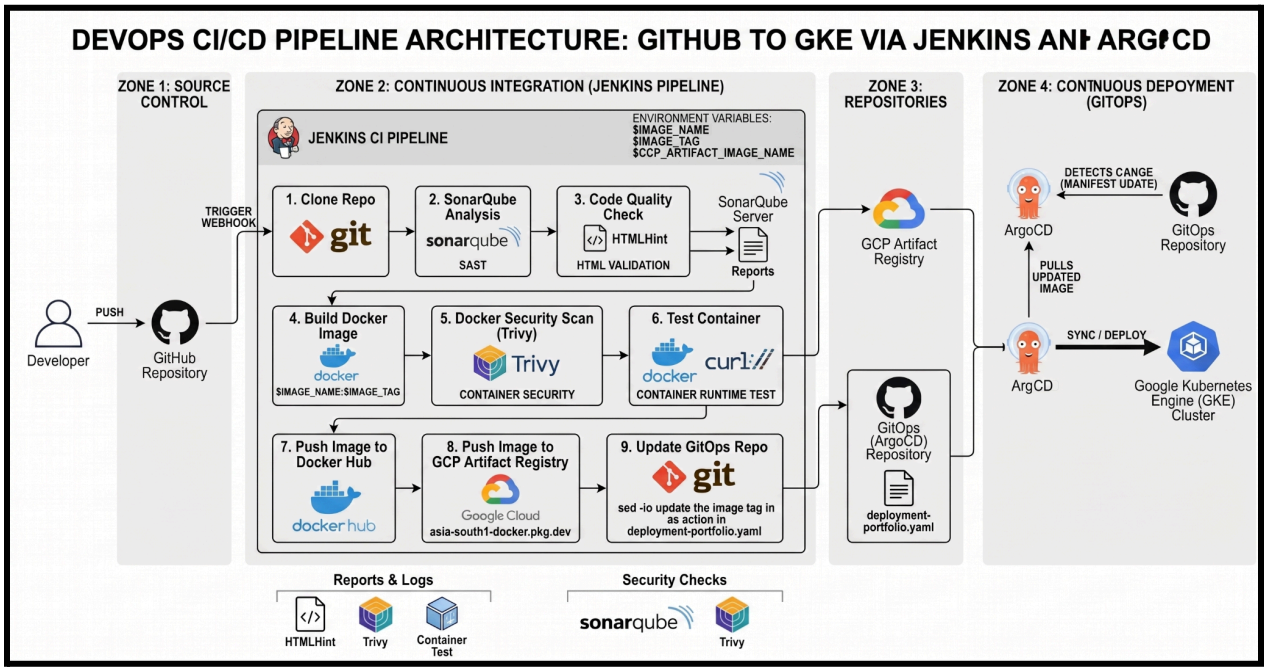
This project demonstrates a complete CI/CD workflow integrating source control, automated pipelines, containerization, cloud services, and GitOps-based Kubernetes deployment.

The workflow ensures:

- Continuous Integration (CI) → Code validation, quality checks, and build
- Continuous Delivery (CD) → Automated deployment using GitOps
- Security Integration → Vulnerability scanning and reporting
- Cloud Deployment → Scalable application hosting on Kubernetes

---

Architecture Diagram (High-Level Flow)



Architecture Components Explanation

Component	Tool Used	Purpose
Source Code Management	GitHub	Stores and manages application code

CI/CD Pipeline	Jenkins	Automates build, test, and deployment process
Code Quality	SonarQube	Analyzes code quality and detects issues
Security Scanning	Trivy	Scans Docker images for vulnerabilities
Containerization	Docker	Packages application into portable containers
Container Registry	Docker Hub / GCP Artifact Registry	Stores Docker images
GitOps	ArgoCD	Automates Kubernetes deployment from Git
Orchestration	GKE (Kubernetes)	Runs and manages containerized applications
Monitoring	Prometheus & Grafana	Observability and metrics visualization

---

Workflow Summary

1. Developer pushes code to GitHub
2. Jenkins pipeline is triggered automatically

3. Code is analyzed (SonarQube) and scanned (Trivy)
4. Docker image is built and pushed to registries
5. GitOps repository is updated with new image version
6. ArgoCD detects changes and deploys to Kubernetes
7. Application is exposed via LoadBalancer
8. Monitoring is enabled using Prometheus and Grafana

Outcome

- **Fully automated CI/CD pipeline**
- **Secure and scalable deployment using Kubernetes**
- **Real-time monitoring and observability**
- **Production-ready DevOps workflow implementation**

2. Source Code Management (GitHub) :-

2.1 Objective

To host and manage application source code using version control and enable CI/CD integration.

2.2 Tool Used

- GitHub

2.3 Steps Performed

1. Create GitHub Account

- Registered a new GitHub account.

2. Create a New Repository

- Repository Name: `portfolio-project`
- Visibility: Public / Private

3. Initialize Local Repository

```
git init
```

4. Add Remote Repository

```
git remote add origin
https://github.com/Anubharanidharan28/portfolio-project.git
```

5. Add Source Code

```
git add .
```

6. Commit Changes

```
git commit -m "Initial commit"
```

7. Push Code to GitHub

```
git push -u origin main
```

2.4 Outcome

- Source code successfully pushed to GitHub repository.
- The repository is ready for CI/CD pipeline integration.

---

3. Jenkins Master Setup (Docker) :-

3.1 Objective

To set up a Jenkins master instance for building and managing CI/CD pipelines.

3.2 Tool Used

- Jenkins
- Docker

3.3 Steps Performed

1. Pull Jenkins Docker Image

```
docker pull jenkins/jenkins:lts
```

---

2. Run Jenkins Container

```
docker run -d --name jenkins-master --network host \
 -p 8080:8080 -p 50000:50000 \
 -v jenkins_home:/var/jenkins_home \
 jenkins/jenkins:lts
```

---

3. Verify Container Status

```
docker ps
```

---

4. Access Jenkins UI

- Open browser:

```
http://<VM-IP>:8080
```

---

5. Get Initial Admin Password

```
docker exec jenkins-master cat
/var/jenkins_home/secrets/initialAdminPassword
```

---

6. Complete Jenkins Setup

- Enter admin password
- Install suggested plugins
- Create admin user

3.4 Key Observations

- Used `--network host` to simplify networking and avoid port mapping issues.
- Mounted volume `jenkins_home` to persist Jenkins data (jobs, plugins, configs).
- Jenkins runs on port **8080** (UI) and **50000** (agent communication).
- Initial setup requires unlocking Jenkins using generated admin password.
- First-time plugin installation may take a few minutes.

3.5 Outcome

- Jenkins master successfully deployed using Docker.
- Jenkins UI accessible via browser.
- Ready to configure pipelines and integrate with other tools.

4. Jenkins Agent Setup (GCP VM) :-

4.1 Objective

To configure a Jenkins agent node on a GCP VM for executing CI/CD pipeline jobs.

4.2 Tools Used

- Jenkins
- Google Cloud Platform (GCP VM)
- SSH

4.3 Steps Performed

1. Create GCP Virtual Machine

- Created a VM instance to act as Jenkins agent.

---

2. Install Required Packages on VM

```
sudo apt update && sudo apt upgrade -y
```

Install Java (Required for Jenkins Agent)

```
sudo apt install openjdk-17-jdk -y
```

Install Docker

```
sudo apt install docker.io -y
sudo usermod -aG docker $USER
newgrp docker
```

Install Node.js and npm

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

---

3. Create SSH Key Pair

```
ssh-keygen -t rsa -b 4096
```

- Generates:
  - Public Key → `~/.ssh/id_rsa.pub`
  - Private Key → `~/.ssh/id_rsa`

---

4. Configure SSH Access

- Open VM:

```
vi ~/.ssh/authorized_keys
```

- Paste **public key** content.

---

5. Set Permissions

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

---

6. Add Credentials in Jenkins

- Go to Jenkins → Manage Jenkins → Credentials
- Add:
  - Kind: SSH Username with Private Key
  - Username: VM username
  - Private Key: Paste content of `id_rsa`

---

7. Configure Jenkins Node (Agent)

- Go to Jenkins → Manage Nodes → New Node
- Configure:
  - Node Name: `GCP-Jenkins-Agent`
  - Type: Permanent Agent
  - Remote Root Directory: `/home/<user>`
  - Launch Method: Launch via SSH
  - Credentials: Select added SSH credentials

4.4 Key Observations

- Java is mandatory for Jenkins agent communication.
- Docker installation allows container build and execution inside pipeline.
- Node.js is required for frontend tools like HTMLHint.
- SSH key-based authentication is more secure than password authentication.
- Correct permissions for `.ssh` directory are required to avoid connection issues.

4.5 Outcome

- Jenkins agent successfully connected to Jenkins master.
- Pipeline jobs can be executed on GCP VM.
- Agent ready for Docker builds, scanning, and deployments.

5. SonarQube Setup and Jenkins Integration :-

5.1 Run SonarQube Container

Deploy SonarQube using Docker on the Jenkins Agent or a dedicated VM:

```
docker run -d \
 --name sonarqube \
 -p 9000:9000 \
 sonarqube:lts-community
```

Post-Setup Configuration

```
sudo usermod -aG docker $USER
newgrp docker
```

5.2 Access SonarQube Dashboard

Open a browser and navigate to:

```
http://<SONARQUBE_SERVER_IP>:9000
```

Default Credentials

- Username: `admin`
- Password: `admin`
  (Change password on first login)

5.3 Create SonarQube Project

Inside SonarQube UI:

- Click **Create Project**
- Select **Manual Project Setup**

Project Details

Field	Value
Project Name	portfolio-website
Project Key	portfolio-website

5.4 Generate Authentication Token

Navigate to:

My Account → Security → Generate Token

Token Details

Field	Value
Token Name	jenkins-sonar-token

- Click **Generate**
- Copy and store the token securely (cannot be retrieved again)

5.5 Configure SonarQube Credentials in Jenkins

Navigate:

Manage Jenkins → Credentials → Global → Add Credentials

Configuration

Field	Value
Kind	Secret Text
Secret	SonarQube Token
ID	Sonar-Tokens
Description	SonarQube Auth Token

5.6 Configure SonarQube Server in Jenkins

Navigate:

Manage Jenkins → System → SonarQube Servers

Server Configuration

Field	Value
Name	SonarQube
Server URL	http://<SONARQUBE_SERVER_IP>:9000
Authentication Token	Sonar-Tokens

5.7 Install Required Jenkins Plugin

Navigate:

Manage Jenkins → Manage Plugins → Available Plugins

Install:

- **SonarQube Scanner for Jenkins**

5.8 Configure Sonar Scanner Tool

Navigate:

Manage Jenkins → Global Tool Configuration

Add Sonar Scanner

Field	Value
Name	sonar-scanner
Type	Install Automatically

5.9 Jenkins Pipeline Integration

SonarQube is integrated into the pipeline using:

```
withSonarQubeEnv('SonarQube') {
  withCredentials([string(credentialsId: 'Sonar-Tokens', variable: 'SONAR_TOKEN')]) {
    sh """
    ${scannerHome}/bin/sonar-scanner \
      -Dsonar.projectKey=portfolio-website \
      -Dsonar.sources=. \
      -Dsonar.host.url=http://<SONARQUBE_SERVER_IP>:9000 \
      -Dsonar.login=$SONAR_TOKEN
    """
  }
}
```

5.10 Key Outcome

- Static code analysis integrated into CI pipeline
- Automated quality checks during build
- Centralized dashboard for code quality metrics
- Enables early detection of bugs, vulnerabilities, and code smells

6. Code Quality & Security Tool Setup :-

Objective

Prepare the Jenkins Agent environment to perform **code quality validation**, **security scanning**, and **basic container testing** as part of the DevSecOps workflow.

6.1 System Preparation

Before installing tools, update the system packages to ensure compatibility and avoid dependency issues.

```
sudo apt update && sudo apt upgrade -y
```

6.2 Install Node.js and npm

Node.js is required to run frontend linting tools such as **HTMLHint**.

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

Verify Installation

```
node -v
npm -v
```

6.3 Install HTMLHint (Code Quality Tool)

HTMLHint is used to validate HTML code quality and detect syntax issues.

```
sudo npm install -g htmlhint
```

Verify Installation

```
htmlhint --version
```

6.4 Install Trivy (Security Scanning Tool)

Trivy is used to scan Docker images for vulnerabilities.

Step 1: Remove Existing Broken Configuration (if any)

```
sudo rm -f /usr/share/keyrings/trivy.gpg
sudo rm -f /etc/apt/sources.list.d/trivy.list
```

Step 2: Add Trivy GPG Key (Secure Method)

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://aquasecurity.github.io/trivy-repo/deb/public.key \
| sudo gpg --dearmor -o /etc/apt/keyrings/trivy.gpg
```

Step 3: Add Trivy Repository

```
echo "deb [signed-by=/etc/apt/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb jammy main" \
| sudo tee /etc/apt/sources.list.d/trivy.list
```

Step 4: Update Package List

```
sudo apt update
```

Step 5: Install Trivy

```
sudo apt install trivy -y
```

Step 6: Verify Installation

```
trivy -v
```

6.5 Key Tools Installed

Tool	Purpose
Node.js	Runtime for JavaScript tools
npm	Package manager for Node.js
HTMLHint	HTML code quality validation
Trivy	Container security vulnerability scanning

6.6 Outcome

After completing this setup:

- The system is ready to perform **code linting** using HTMLHint
- Docker images can be scanned for **security vulnerabilities** using Trivy
- The Jenkins agent is fully prepared for **DevSecOps pipeline execution**

6.7 Important Notes

- Always install tools on the **Jenkins Agent**, not just the master
- Ensure Docker is installed and running before using Trivy
- Use updated repositories to avoid installation failures

- Validate tool installations before integrating into CI/CD workflows

7. Docker Image Build & Tagging :-

Objective

Build a Docker image from the application source code and tag it properly for versioning and deployment.

7.1 Prerequisites

Before building the Docker image, ensure the following:

- Docker is installed on the Jenkins Agent
- Docker service is running
- User has permission to run Docker commands

Add User to Docker Group (if not already done)

```
sudo usermod -aG docker $USER
newgrp docker
```

7.2 Verify Docker Installation

```
docker --version
docker info
```

7.3 Dockerfile Requirement

Ensure your project contains a valid `Dockerfile` in the root directory.

Example (Simple Nginx-based Portfolio Website)

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
```

7.4 Build Docker Image

Build the Docker image using the project source code.

```
docker build -t anubharani/personal-portfolio-website:<TAG> .
```

> Replace `<TAG>` with a version or build number (e.g., `1`, `latest`, etc.)

7.5 Tag Docker Image

Tag the image for multiple purposes such as versioning and pushing to registries.

docker tag anubharani/personal-portfolio-website:<TAG>
anubharani/personal-portfolio-website:latest

Tag for Google Artifact Registry
docker tag anubharani/personal-portfolio-website:<TAG>
asia-south1-docker.pkg.dev/testing-bharani/personal-portfolio-website/portfolio-website:<TAG>

docker tag anubharani/personal-portfolio-website:<TAG>
asia-south1-docker.pkg.dev/testing-bharani/personal-portfolio-website/portfolio-website:latest

---

7.6 Verify Built Images

docker images

You should see:

- `anubharani/personal-portfolio-website:<TAG>`
- `anubharani/personal-portfolio-website:latest`
- GCP Artifact Registry tagged images

7.7 Outcome

After completing this step:

- Docker image is successfully built from source code
- Image is properly tagged for:
  - Version tracking
  - Latest deployment reference

    - Cloud registry usage

7.8 Important Notes

- Always use meaningful tags (e.g., build number or version)
- Keep `latest` tag for quick deployments
- Ensure Dockerfile is optimized to reduce image size
- Validate build before pushing to registries

8. Push Docker Image to Docker Hub :-

Objective

Push the built Docker image to Docker Hub so it can be stored centrally and used for deployments.

8.1 Create Docker Hub Account

- Go to Docker Hub
- Create an account (if not already created)
- Login to your account

8.2 Create Docker Repository

Inside Docker Hub:

- Click **Create Repository**
- Repository Name: `personal-portfolio-website`
- Visibility: Public / Private (based on requirement)
- Click **Create**

8.3 Verify Repository Naming Convention

Your image name must match the Docker Hub format:

<docker-username>/<repository-name>:<tag>

Example:

anubharani/personal-portfolio-website:1
anubharani/personal-portfolio-website:latest

8.4 Create Docker Hub Access Token (Recommended)

Instead of using your password:

- Go to **Account Settings** → **Security** → **Access Tokens**
- Click **Generate Token**
- Name: `jenkins-docker-token`
- Copy the token (used as password in Jenkins)

8.5 Add Docker Hub Credentials in Jenkins

Navigate in Jenkins:

- **Manage Jenkins** → **Credentials** → **Global** → **Add Credentials**

Configuration:

Field	Value
Kind	Username with Password
Username	Docker Hub Username
Password	Docker Hub Access Token
ID	dockerhub-creds
Description	Docker Hub Credentials

8.6 Authenticate Docker from Jenkins Agent

Ensure Docker login is performed securely using credentials.

```
echo <DOCKER_PASSWORD> | docker login -u <DOCKER_USERNAME> --password-stdin
```

8.7 Push Docker Image

Push both versioned and latest images:

```
docker push anubharani/personal-portfolio-website:<TAG>
docker push anubharani/personal-portfolio-website:latest
```

8.8 Verify Image in Docker Hub

- Go to Docker Hub repository
- Refresh the page
- Confirm:
    - Image tags are visible
    - Latest and version tags are updated

8.9 Outcome

After completing this step:

- Docker images are successfully stored in Docker Hub
- Images are accessible from anywhere
- Ready for deployment in Kubernetes or other platforms

8.10 Important Notes

- Always use **Access Tokens**, not your Docker password
- Ensure correct repository naming (`username/repo`)
- Tag images properly before pushing
- Verify login before pushing images
- Keep repository naming consistent with pipeline configuration

9. Push Docker Image to GCP Artifact Registry :-

Objective

Store Docker images securely in Google Cloud for use in Kubernetes deployments (GKE) and production environments.

9.1 Create Artifact Registry Repository

In GCP Console:

- Navigate to **Artifact Registry**
- Click **Create Repository**

Configuration:

Field	Value
Name	personal-portfolio-website
Format	Docker
Region	asia-south1
Description	Docker images for portfolio

👉 Click **Create**

9.2 Create Service Account (SA)

Navigate:
IAM & Admin → Service Accounts

- Click **Create Service Account**

Configuration:

Field	Value

Name	artifact-registry-sa
ID	artifact-registry-sa

9.3 Assign Permissions to Service Account

Grant the following role:

Role	Purpose
Artifact Registry Writer	Push images to registry

(Optional: **Artifact Registry Admin** for full control)

9.4 Generate Service Account Key (JSON)

- Open created Service Account
- Go to **Keys** → **Add Key** → **Create New Key**
- Select **JSON**
- Download the file

👉 This JSON file will be used by Jenkins for authentication.

9.5 Add GCP Credentials in Jenkins

Navigate:
Manage Jenkins → **Credentials** → **Global** → **Add Credentials**

Configuration:

Field	Value
Kind	Secret File
File	Upload JSON file
ID	gcp-service-account-json
Description	GCP Artifact Registry Access

9.6 Authenticate with GCP

```
gcloud auth activate-service-account --key-file=<PATH_TO_JSON>
```

9.7 Configure Docker for Artifact Registry

```
gcloud auth configure-docker asia-south1-docker.pkg.dev -q
```

9.8 Tag Docker Image for GCP

Format:

```
asia-south1-docker.pkg.dev/<PROJECT_ID>/personal-portfolio-website/portfolio-website:<TAG>
```

Example:

```
asia-south1-docker.pkg.dev/testing-bharani/personal-portfolio-website/portfolio-website:1
```

9.9 Push Docker Image

```
docker push
asia-south1-docker.pkg.dev/<PROJECT_ID>/personal-portfolio-website/portfoli
o-website:<TAG>
```

Example:

```
docker push
asia-south1-docker.pkg.dev/<PROJECT_ID>/personal-portfolio-website/portfoli
o-website:latest
```

9.10 Verify Image in GCP

- Go to **Artifact Registry** → **Repository**
- Open your repository

Check:

- Image names
- Tags (latest, version)
- Push timestamps

9.11 Outcome

After completing this step:

- Docker images are securely stored in GCP
- Images are ready for deployment in GKE
- Authentication is automated via Jenkins credentials

9.12 Important Notes

- Always use **Service Account JSON**, not personal login
- Follow **least privilege principle**
- Ensure correct region (**asia-south1**)
- Match image name exactly with repository structure

- Run `gcloud auth configure-docker` before pushing

10. GKE Cluster Setup & ArgoCD GitOps Deployment :-

Objective

Provision a Kubernetes cluster in GCP, install ArgoCD, integrate GitHub securely using Jenkins credentials, and implement GitOps-based deployment.

10.1 Create and Connect to GKE Cluster

Provision a Kubernetes cluster in **Google Kubernetes Engine** and connect via Cloud Shell.

```
gcloud container clusters get-credentials portfolio-kubernetes-cluster \
--region asia-south1-a
```

Verify Cluster Access:

```
kubectl get nodes
```

10.2 Install ArgoCD in Kubernetes

Create Namespace:

```
kubectl create namespace argocd
```

Install ArgoCD Components from **Argo CD**:

```
kubectl apply -n argocd -f \
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install
.yaml
```

10.3 Expose ArgoCD Server (LoadBalancer)

```
kubectl patch svc argocd-server -n argocd \
-p '{"spec": {"type": "LoadBalancer"}}'
```

10.4 Access ArgoCD UI

Get External IP:

```
kubectl get svc argocd-server -n argocd
```

Get Admin Password:

```
kubectl get secret argocd-initial-admin-secret -n argocd \
-o jsonpath="{.data.password}" | base64 -d
```

Login Details:

Field	Value
Username	admin
Password	Retrieved from above command

Open in browser:

http://<EXTERNAL-IP>

10.5 Create GitOps Repository (GitHub)

Create a repository in **GitHub**
Example: `Gitops-argoCD-repo`

Add Kubernetes manifests:

- Deployment YAML
- Service YAML
- Ingress YAML

Push Code:

```
git init
git remote add origin <GITOPS_REPO_URL>
git add .
git commit -m "Initial GitOps commit"
git push -u origin main
```

10.6 Configure GitHub Credentials in Jenkins

Create GitHub Personal Access Token (PAT):

Go to:
Settings → Developer Settings → Personal Access Tokens

Permission	Purpose
repo	Full repository access

Add Credentials in **Jenkins**:

Field	Value
Kind	Username/Password

Username	GitHub Username
Password	GitHub PAT
ID	github-creds
Description	GitHub Access

10.7 Connect GitHub Repository to ArgoCD

In ArgoCD UI:

- Go to **Settings** → **Repositories**
- Click **Connect Repo**
- Provide:
  - Repository URL
  - Authentication (if private)

10.8 Create ArgoCD Application

In ArgoCD UI:

Click **New App**

Field	Value
Application Name	portfolio-app
Project	default

Repository URL	GitOps repo URL
Revision	main
Path	Folder containing YAML files
Cluster URL	https://kubernetes.default.svc
Namespace	default / required namespace
Sync Policy	Automatic (recommended)

👉 Click **Create**

10.9 Sync Application

Click **Sync** in ArgoCD UI

ArgoCD will:

- Pull manifests from GitHub
- Deploy resources to GKE cluster

10.10 ArgoCD CLI Login (Optional)

```
argocd login <EXTERNAL-IP> \
--username admin \
--password <PASSWORD> \
--insecure
```

10.11 End-to-End GitOps Flow

```
Developer → GitHub → Jenkins → Docker Image → Registry
       → GitOps Repo Update → ArgoCD → GKE Deployment
```

10.12 Outcome

After completing this step:

- GKE cluster is operational
- ArgoCD is installed and accessible
- GitHub repo integrated securely
- Jenkins updates GitOps repo automatically
- Continuous deployment enabled via GitOps

10.13 Important Notes

- Use **PAT instead of GitHub password**
- Store credentials securely in Jenkins
- Never hardcode credentials in scripts
- Enable **Auto Sync** for full GitOps automation
- Change ArgoCD default admin password after login
- Ensure correct namespace and repo path

11. Jenkins CI/CD Pipeline – End-to-End DevSecOps Workflow :-

Objective

Automate the complete software delivery lifecycle including code checkout, quality analysis, security scanning, containerization, registry push, and GitOps deployment update.

11.1 Pipeline Overview

This **Jenkins** Declarative Pipeline performs:

- Source Code Checkout
- Static Code Analysis (**SonarQube**)
- Code Quality Validation (**HTMLHint**)
- Docker Image Build
- Security Scanning (**Trivy**)
- Container Testing
- Report Generation & Archiving
- Push to **Docker Hub**
- Push to **Google Artifact Registry**
- Update GitOps Repository (**Argo CD** trigger)

11.2 Pipeline Definition (Groovy Script)

```
pipeline {
  agent { label 'GCP-Jenkins-Agent' }

  environment {
      IMAGE_NAME = "anubharani/personal-portfolio-website"
      IMAGE_TAG = "${BUILD_NUMBER}"
      GCP_ARTIFACT_IMAGE_NAME =
"asia-south1-docker.pkg.dev/testing-bharani/personal-portfolio-website/port
folio-website"
  }

  stages {

      stage('Clone Repo') {
          steps {
              git branch: 'main', url:
'https://github.com/Anubharanidharan28/portfolio-project.git'
          }
      }

      stage('SonarQube Analysis') {
          steps {
              script {
                  def scannerHome = tool 'sonar-scanner'

                  withSonarQubeEnv('SonarQube') {
                      withCredentials([string(credentialsId:
'Sonar-Tokens', variable: 'SONAR_TOKEN')]) {
```

```
                            sh """
                            ${scannerHome}/bin/sonar-scanner \
                                -Dsonar.projectKey=portfolio-website \
                                -Dsonar.sources=. \
                                -Dsonar.host.url=http://34.14.154.234:9000 \
                                -Dsonar.login=$SONAR_TOKEN
                            """
                        }
                    }
                }
            }
        }

        stage('Code Quality Check') {
            steps {
                sh '''
                mkdir -p reports
                htmlhint index.html > reports/html-report.txt || true
                '''
            }
        }

        stage('Build Docker Image') {
            steps {
                sh """
                docker build -t $IMAGE_NAME:$IMAGE_TAG .
                docker tag $IMAGE_NAME:$IMAGE_TAG $IMAGE_NAME:latest
                docker tag $IMAGE_NAME:$IMAGE_TAG
$GCP_ARTIFACT_IMAGE_NAME:$IMAGE_TAG
                docker tag $IMAGE_NAME:$IMAGE_TAG
$GCP_ARTIFACT_IMAGE_NAME:latest
                """
            }
        }

        stage('Docker Security Scan') {
            steps {
                sh '''
                trivy image --format json -o reports/trivy-report.json
$IMAGE_NAME:$IMAGE_TAG
                trivy image --severity HIGH,CRITICAL $IMAGE_NAME:$IMAGE_TAG >
reports/trivy-summary.txt || true
                '''
```

```
        }
    }

    stage('Test Container') {
        steps {
            sh '''
            docker run -d -p 8085:80 --name test-container
$IMAGE_NAME:$IMAGE_TAG
            sleep 10
            curl -I http://localhost:8085 > reports/container-test.txt ||
true
            docker rm -f test-container
            '''
        }
    }

    stage('Generate DevSecOps Report') {
        steps {
            sh '''
            echo "==============================" >
reports/devsecops-report.txt
            echo " DEVSECOPS PIPELINE REPORT" >>
reports/devsecops-report.txt
            echo "==============================" >>
reports/devsecops-report.txt
            echo "" >> reports/devsecops-report.txt
            echo "BUILD NUMBER: $BUILD_NUMBER" >>
reports/devsecops-report.txt
            echo "DATE: $(date)" >> reports/devsecops-report.txt

            echo "" >> reports/devsecops-report.txt
            echo "----- CODE QUALITY CHECK (HTMLHint) -----" >>
reports/devsecops-report.txt
            cat reports/html-report.txt >> reports/devsecops-report.txt

            echo "" >> reports/devsecops-report.txt
            echo "----- DOCKER SECURITY SCAN (Trivy) -----" >>
reports/devsecops-report.txt
            cat reports/trivy-summary.txt >> reports/devsecops-report.txt

            echo "" >> reports/devsecops-report.txt
            echo "----- CONTAINER TEST -----" >>
reports/devsecops-report.txt
```

```
                cat reports/container-test.txt >>
reports/devsecops-report.txt
                '''
            }
        }

        stage('Archive Reports') {
            steps {
                archiveArtifacts artifacts: 'reports/*', fingerprint: true
            }
        }

        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(credentialsId:
'dockerhub-creds', usernameVariable: 'DOCKER_USER', passwordVariable:
'DOCKER_PASS')]) {
                    sh """
                    echo $DOCKER_PASS | docker login -u $DOCKER_USER
--password-stdin
                    docker push $IMAGE_NAME:$IMAGE_TAG
                    docker push $IMAGE_NAME:latest
                    """
                }
            }
        }

        stage('Push to GCP Artifact Registry') {
            steps {
                withCredentials([file(credentialsId:
'gcp-service-account-json', variable: 'GCP_KEY')]) {
                    sh """
                    gcloud auth activate-service-account --key-file=$GCP_KEY
                    gcloud auth configure-docker asia-south1-docker.pkg.dev
-q
                    docker push $GCP_ARTIFACT_IMAGE_NAME:$IMAGE_TAG
                    docker push $GCP_ARTIFACT_IMAGE_NAME:latest
                    """
                }
            }
        }

        stage('Update GitOps Repo') {
```

```
        steps {
            withCredentials([
                usernamePassword(
                    credentialsId: 'github-creds',
                    usernameVariable: 'GIT_USER',
                    passwordVariable: 'GIT_PASS'
                )
            ]) {
                sh '''
                    rm -rf gitops-repo
                    git clone
https://${GIT_USER}:${GIT_PASS}@github.com/Anubharanidharan28/Gitops-argoCD
-repo.git gitops-repo
                    cd gitops-repo

                    sed -i "s|image: .*|image:
'"$GCP_ARTIFACT_IMAGE_NAME:$IMAGE_TAG"'|g" deployment-portfolio.yaml

                    git config user.email "anubharanidharan28@gmail.com"
                    git config user.name "Anubharanidharan M"

                    git add deployment-portfolio.yaml
                    git commit -m "Update image to version ${IMAGE_TAG}"
|| echo "No changes to commit"
                    git push origin main
                '''
            }
        }
    }
  }
}
```

11.3 Stage-by-Stage Explanation

1. **Clone Repo**
   Pulls latest code from **GitHub**
2. **SonarQube Analysis**
   Detects bugs, vulnerabilities, code smells
3. **Code Quality Check**
   Validates frontend code using HTMLHint

4. **Build Docker Image**
   Creates versioned + latest images
5. **Docker Security Scan**
   Uses Trivy to detect vulnerabilities
6. **Test Container**
   Ensures app runs correctly before deployment
7. **Generate DevSecOps Report**
   Combines all reports into one file
8. **Archive Reports**
   Stores reports in Jenkins
9. **Push to Docker Hub**
   Publishes image publicly
10. **Push to GCP Artifact Registry**
    Stores image securely for Kubernetes
11. **Update GitOps Repository**
    Updates image tag → triggers ArgoCD deployment

11.4 End-to-End Flow

Developer → GitHub → Jenkins → Build → Scan → Push → GitOps Repo → ArgoCD → GKE

11.5 Key Features

Feature	Description
Automation	Fully automated CI/CD
Security	Integrated vulnerability scanning
Quality	SonarQube + HTMLHint
Versioning	Build number tagging

GitOps	ArgoCD-based deployment
Multi Registry	Docker Hub + GCP Artifact Registry
Reporting	Centralized DevSecOps reports

11.6 Important Notes

- Store credentials securely in Jenkins
- Never hardcode secrets
- Ensure tools installed on agent:
    - Docker
    - Trivy
    - Node.js
    - Sonar Scanner
- Maintain consistent image naming
- Use GitOps repo as **single source of truth**

11.7 Outcome

After pipeline execution:

- Application is built, tested, and secured
- Images pushed to registries
- GitOps repo updated automatically
- **Argo CD** deploys latest version to Kubernetes

12. Monitoring Setup – Prometheus & Grafana using Helm (GKE) :-

Objective

Deploy a full monitoring stack using Helm in a Kubernetes (GKE) cluster, expose services externally, and visualize metrics using Grafana dashboards integrated with Prometheus.

12.1 Install Helm in Cloud Shell

Verify Helm Installation

```
helm version
```

If not installed:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

12.2 Add Helm Repositories

Add Prometheus Community Repo

```
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
```

Add Grafana Repo (Optional but recommended)

```
helm repo add grafana https://grafana.github.io/helm-charts
```

Update Repositories

```
helm repo update
```

12.3 Create Monitoring Namesp

```
kubectl create namespace monitoring
```

12.4 Install kube-prometheus-stack

This Helm chart includes:

- Prometheus

- Grafana
- Alertmanager
- Node Exporter
- kube-state-metrics

Install with LoadBalancer Exposure

```
helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--set grafana.service.type=LoadBalancer \
--set prometheus.service.type=LoadBalancer
```

12.5 Verify Deployment

```
kubectl get pods -n monitoring
```

Ensure all pods are in **Running** state.

12.6 Expose Services (LoadBalancer)

```
kubectl get svc -n monitoring
```

Look for:

Service	Purpose
monitoring-kube-prometheus-grafana	Grafana UI
monitoring-kube-prometheus-prometheus	Prometheus UI

12.7 Access Grafana

Get Grafana Password

```
kubectl get secret monitoring-grafana -n monitoring \
-o jsonpath="{.data.admin-password}" | base64 -d
```

Login Details

Field	Value
URL	http://<GRAFANA-EXTERNAL-IP>
Username	admin
Password	Retrieved from above command

12.8 Access Prometheus

Open in browser:

```
http://<PROMETHEUS-EXTERNAL-IP>
```

12.9 Understanding Components

Component	Purpose
Prometheus	Metrics collection & storage
Grafana	Visualization dashboards

Alertmanager	Alert handling
Exporters	Collect system & Kubernetes metrics

12.10 Configure Grafana Data Source

Steps in Grafana UI:

- Go to **Settings → Data Sources**
- Click **Add Data Source**
- Select **Prometheus**

Configure:

Field	Value
Name	Prometheus
URL	http://monitoring-kube-prometheus-prometheus.monitoring.svc:9090

Click **Save & Test**

12.11 Connect Prometheus to Grafana

Grafana queries Prometheus using internal Kubernetes service.
No external exposure required for internal communication.

12.12 Using Prometheus

Example Queries:

CPU Usage

rate(container_cpu_usage_seconds_total[5m])

Memory Usage

container_memory_usage_bytes

Pod Count

kube_pod_info

12.13 Create Grafana Dashboard

Steps:

- Open Grafana UI
- Click **+** → **Dashboard**
- Click **Add New Panel**
- Select **Prometheus Data Source**

Example Query

rate(container_cpu_usage_seconds_total[5m])

Configure:

- Visualization (Graph, Table, Gauge)
- Panel Title

Click **Apply** → Save Dashboard

12.14 Import Pre-built Dashboards (Recommended)

Steps:

- Go to **+** → **Import**

Use Dashboard IDs:

Dashboard	ID

Kubernetes Cluster Monitoring	315
Node Exporter Full	1860

Select Prometheus Data Source → Click **Import**

12.15 Verify Monitoring Setup

- Check metrics in Prometheus UI
- Verify dashboards in Grafana
- Ensure real-time data updates

12.16 End-to-End Monitoring Flow

Kubernetes cluster generates metrics
→ Prometheus scrapes metrics
→ Data stored in Prometheus TSDB
→ Grafana queries Prometheus
→ Dashboards visualize metrics

12.17 Important Notes

- Use LoadBalancer only for external access
- Secure Grafana with strong password
- Prefer internal service for Prometheus communication
- Monitor resource usage of monitoring stack
- Use persistent storage for production setups

12.18 Outcome

After completing this setup:

- Full monitoring stack is deployed in Kubernetes
- Prometheus collects cluster metrics

- Grafana provides visualization dashboards
- Real-time monitoring is enabled
- System observability is fully established

13. Conclusion

This project demonstrates a **complete, production-grade DevOps and Cloud-native workflow**, integrating CI/CD, DevSecOps, GitOps, and Kubernetes-based monitoring into a unified system.

Starting from **source code management**, the application lifecycle is fully automated using a structured pipeline. Code is continuously validated through quality checks and security scans, containerized using Docker, and stored in both Docker Hub and GCP Artifact Registry. The integration of automated versioning ensures traceability and consistency across deployments.

The adoption of a **GitOps model using ArgoCD** establishes a single source of truth for infrastructure and application state. Any change pushed to the Git repository is automatically synchronized with the Kubernetes cluster, ensuring consistency, auditability, and rapid deployment without manual intervention.

The deployment is hosted on a **Google Kubernetes Engine (GKE)** cluster, providing scalability, high availability, and cloud-native orchestration capabilities. The use of declarative manifests further strengthens infrastructure reliability and repeatability.

To enhance observability, a **robust monitoring stack using Prometheus and Grafana** is implemented. Prometheus continuously collects metrics from the cluster and applications, while Grafana transforms these metrics into meaningful visual dashboards. This enables real-time monitoring, proactive issue detection, and performance optimization.

Overall, this solution achieves:

- **End-to-End Automation** – From code commit to production deployment
- **Security Integration** – Built-in vulnerability scanning and quality gates
- **Scalability & Reliability** – Kubernetes-based deployment on GKE
- **GitOps-Based Delivery** – Automated, version-controlled deployments
- **Observability & Monitoring** – Real-time insights using Prometheus and Grafana

This architecture aligns with **modern DevOps best practices** and reflects a real-world, enterprise-level implementation of cloud-native application delivery. It provides a strong foundation for extending into advanced capabilities such as multi-cloud deployments, service mesh integration, and advanced alerting systems.

Final Outcome:
A fully automated, secure, scalable, and observable DevOps pipeline capable of delivering applications efficiently in a cloud-native environment 🚀